Listas Lineares

Prof. Ricardo Mesquita

Lista Linear

- É uma estrutura que permite armazenar e controlar um conjunto de elementos.
  - Normalmente, os elementos são de mesmo tipo.
- Possui uma ordem: é uma estrutura indexada.
- Podemos utilizar *alocação contígua* (quando conhecemos o tamanho da lista) ou *alocação encadeada*.

Lista Linear

- Operações sobre listas:
  - Inicializar
  - Inserir elemento
  - Listar elementos
  - Buscar elemento
  - Remover elemento
  - Substituir elemento
  - Ordenar a lista
  - Etc.

Lista Linear

- Para alocação contígua, podemos implementar a lista usando vetores.
- A lista admite inserção e remoção em qualquer de suas posições.
- Como as operações sobre listas são muito fundamentais, vamos usar o recurso específico do Python para manipular listas.

Listas em Alocação Contígua

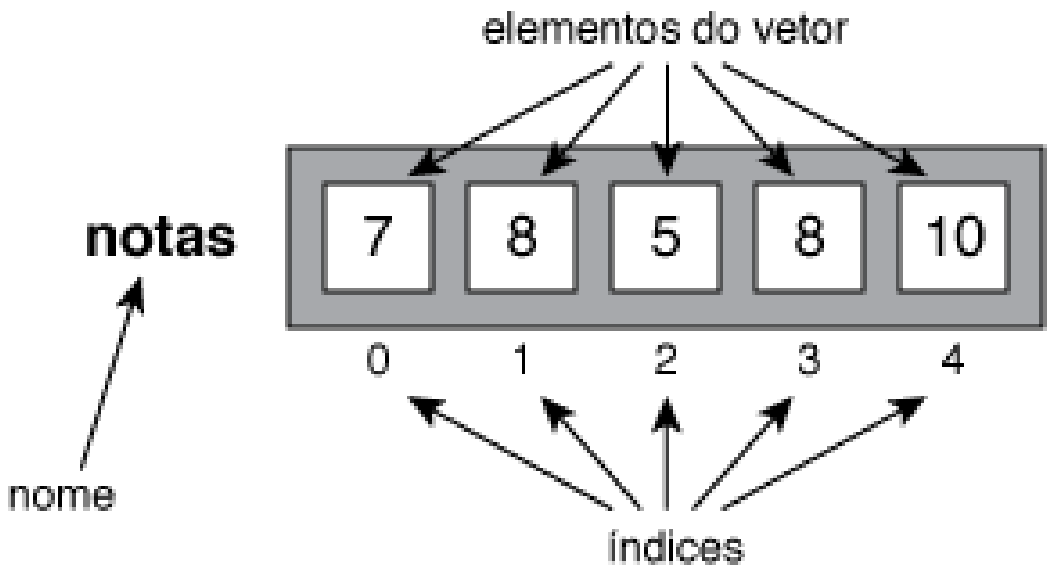
Vetores:

- Um **vetor** é uma coleção de variáveis *de mesmo tipo (homogêneo)* que compartilham uma mesma referência.
  - Declaração (em Python):

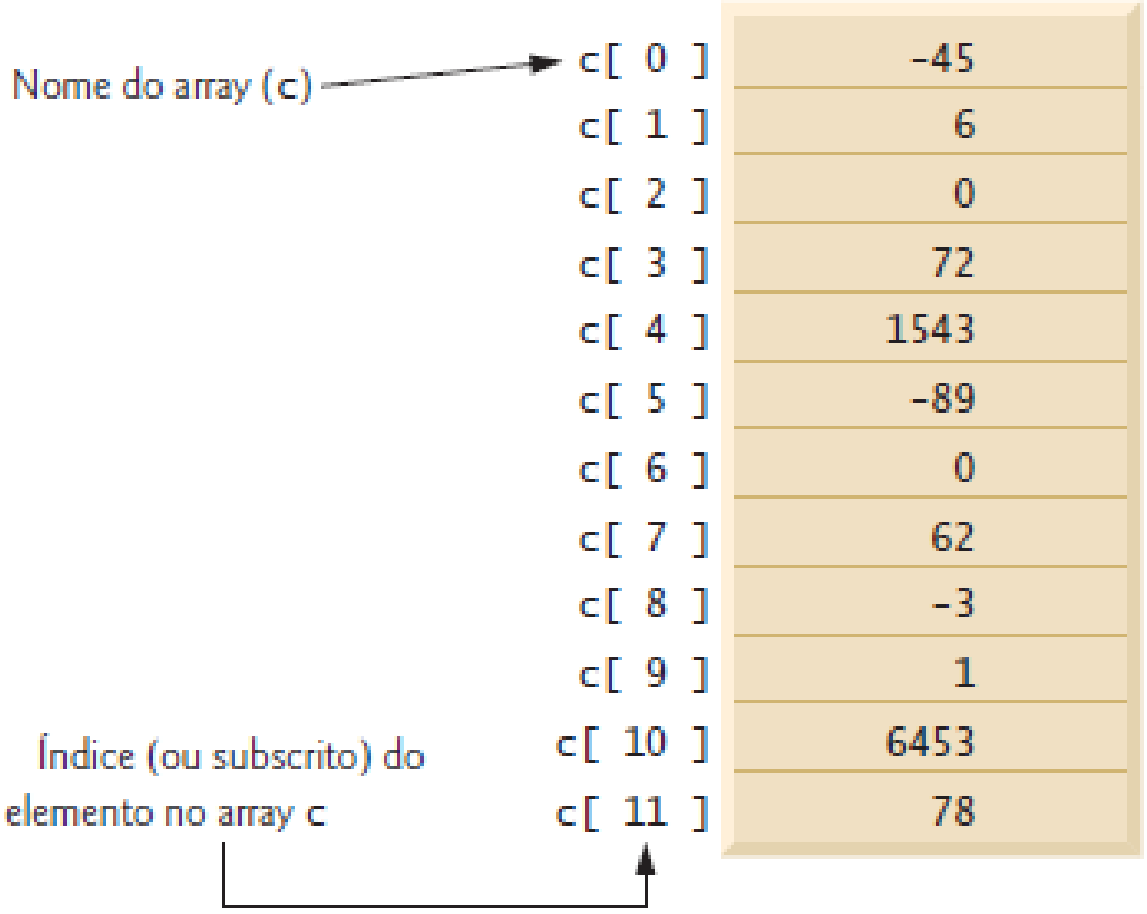
    nome = [<lista de itens>]

  - Exemplo:

    notas = [5.1, 6.3, 4.8, 7.9, 8.5]



Visão Lógica de Vetores



Listas em Python

Declaração e formas de percurso:

- É uma estrutura representada, em Python, *entre colchetes*

```python
lista = ["Pedro", "André", "Matheus", "João", "Tiago"]

for i in lista:
    print(i)

for i in range(len(lista)):
    print(lista[i])

i = 0
while i < len(lista):
    print(lista[i])
    i += 1
```

Listas em Python

Esta ***não é*** uma característica comum nas linguagens de programação!

- Em Python a lista admite *tipos variados*.
  - Exemplo:

```
lista = ["André", '(11)99999-8888', 30]
print(lista)
```

- Pode ser instanciada chamando **list** (o *construtor* de um objeto list):
  - Exemplo:

```
lista = list(("André", '(11)99999-8888', 30))
print(lista)
```

  - Saída para ambos os exemplos: `['André', '(11)99999-8888', 30]`

Listas em Python

- É uma estrutura ***ordenada***. (Ou seja, é ***indexada***.)
  - *Exemplo*:

```
L1 = ["Matheus", "Pedro", "João"]
L2 = ["Pedro", "João", "Matheus"]
print(L1 == L2)
```

Saída: **false**

Listas em Python

- É uma estrutura ***mutável***, ou seja, admite que seus elementos sejam alterados.
  - *Exemplo*:

```python
L1 = ["Matheus", "Pedro", "João"]
print(L1)
L1[2] = "Tiago"
print(L1)
```

Saída:

```
['Matheus', 'Pedro', 'João']
['Matheus', 'Pedro', 'Tiago']
```

Listas em Python

- É uma estrutura que admite elementos duplicados.
  - *Exemplo*:

```
L1 = ["Matheus", "Pedro", "João"]
print(L1)
L1[1] = L1[2] = "Matheus"
print(L1)
```

Saída:

```
['Matheus', 'Pedro', 'João']
['Matheus', 'Matheus', 'Matheus']
```

Operações sobre Listas: Inserção de Elementos

- Inserindo elementos com os métodos **append** e **insert**:

```
lista = ["Pedro", "Matheus", "João"]
lista.append("Tiago")
print(lista)              #['Pedro', 'Matheus', 'João', 'Tiago']
lista.insert(1, "André")
print(lista)              #['Pedro', 'André', 'Matheus', 'João', 'Tiago']
```

Insere no final da lista

Insere na posição indicada pelo argumento

Concatenação de Listas

- Concatenando listas com o método **extend**:

```
lista = ["Pedro", "André", "Matheus", "João"]
outros = ["Tiago", "Felipe", "Simão"]
lista.extend(outros)
print(lista)    #['Pedro', 'André', 'Matheus', 'João', 'Tiago', 'Felipe', 'Simão']
```

Formas de Acessar Elementos de uma Lista

- Acessando elementos de uma lista:

```
lista = [1, 3, 5, 7, 9, 11, 13]
print(lista)            # [1, 3, 5, 7, 9, 11, 13]
print(lista[1])         # 3
print(lista[2:5])       # [5, 7, 9]
print(lista[3:4])       # [7]
print(lista[3:3])       # []
print(lista[:4])        # [1, 3, 5, 7]
print(lista[4:])        # [9,11,13]
```

Formas de Acessar Elementos de uma Lista

- Acessando elementos de uma lista:

```
lista = [1, 3, 5, 7, 9, 11, 13]
print(lista[-4:-1])     # [7, 9, 11]
print(lista[-5:-3])     # [5, 7]
print(lista[-3:-5])     # []
print(lista[-4:-1][2])  # 11
print(lista[::])        # [1, 3, 5, 7, 9, 11, 13]
print(lista[::2])       # [1, 5, 9, 13]
print(lista[::-1])      # [13, 11, 9, 7, 5, 3, 1]
print(lista[::-2])      # [13, 9, 5, 1]
print(lista[::-1][1:4]) # [11, 9, 7]
```

Formas de Acessar Elementos de uma Lista

- Acessando elementos de uma lista:

```
lista = [1, 3, 5, 7, 9, 11, 13]
print(lista[1:5:1])          #[3, 5, 7, 9]
print(lista[1:5:2])          #[3, 7]
print(lista[::-1][1:5:2])    #[11, 7]
print(lista[6:2:-1])         #[13, 11, 9, 7]
print(lista[4::-1])          #[9, 7, 5, 3, 1]
print(lista[:2:-1])          #[13, 11, 9, 7]
```

Usos da Função **len()**

- Função **len**

```
lista = [1, [2, 3], [4, [5, 6]], [7, [8, [9, 10]]]]
print(lista, len(lista))                # [1, [2, 3], [4, [5, 6]], [7, [8, [9, 10]]]] 4
print(lista[2][1], len(lista[2][1]))    # [5, 6] 2
print(lista[3][1][1][0])                # 9
print(len(lista[3][1][1]))              # 2
```

Mais Formas de Manipulação de Listas

- Manipulando listas:

```
lista = [1, 3, 5, 7, 9, 11, 13]
lista[1:3] = [2, 4, 6]      #insere a subsequência
lista[4] = [8]              #substitui o conteúdo por uma lista
lista[5] = 10               #substitui o conteúdo por um novo valor
print(lista)                #[1, 2, 4, 6, [8], 10, 11, 13]
```

Remoção de Elementos

- Removendo elementos com os métodos **remove** e **pop**:

Busca e remove o elemento passado como argumento

```
lista = ["Pedro", "André", "Matheus", "João", "Tiago"]
lista.remove("Matheus")
print(lista)    # ['Pedro', 'André', 'João', 'Tiago']
lista.pop(2)    # remove o elemento da posição 2
print(lista)    # ['Pedro', 'André', 'Tiago']
lista.pop()     # remove o elemento da última posição
print(lista)    # ['Pedro', 'André']
```

Remoção de Elementos

- Método **clear** e comando **del**:

```
lista = ["Pedro", "André", "Matheus", "João", "Tiago"]
del lista[1]          # apaga o elemento da posição 1
print(lista)          # ['Pedro', 'Matheus', 'João', 'Tiago']
del lista[len(lista)-2]
print(lista)          # ['Pedro', 'Matheus', 'Tiago']
lista.clear()         # apaga toda a lista
print(lista)          #[]
```

Busca Sequencial

- Percorre-se a lista, elemento a elemento, e procura-se localizar o elemento solicitado.

- *Exercício*:

  Implemente uma função para realizar uma busca linear em uma lista qualquer.

Busca Linear

- Solução simples:

```
lista = ["Pedro", "André", "Matheus", "Tiago", "Felipe"]
b = False
nome = input("Nome a ser localizado: ")
for i in range(len(lista)):
    if (lista[i] == nome):
        b = True
        print(nome,"é o",i+1,"º elemento da lista")
if (b == False):
    print(nome,"não consta na lista!")
```

Classificação de Listas

- Classificando listas:

```
lista = ["Pedro", "André", "Matheus", "João", "Tiago"]
lista.sort()
print(lista)            # ['André', 'João', 'Matheus', 'Pedro', 'Tiago']
lista.sort(reverse = True)
print(lista)            # ['Tiago', 'Pedro', 'Matheus', 'João', 'André']
```

Vamos estudar diversos algoritmos de classificação.
O método **sort()** é uma implementação do algoritmo QuickSort.

Busca Binária

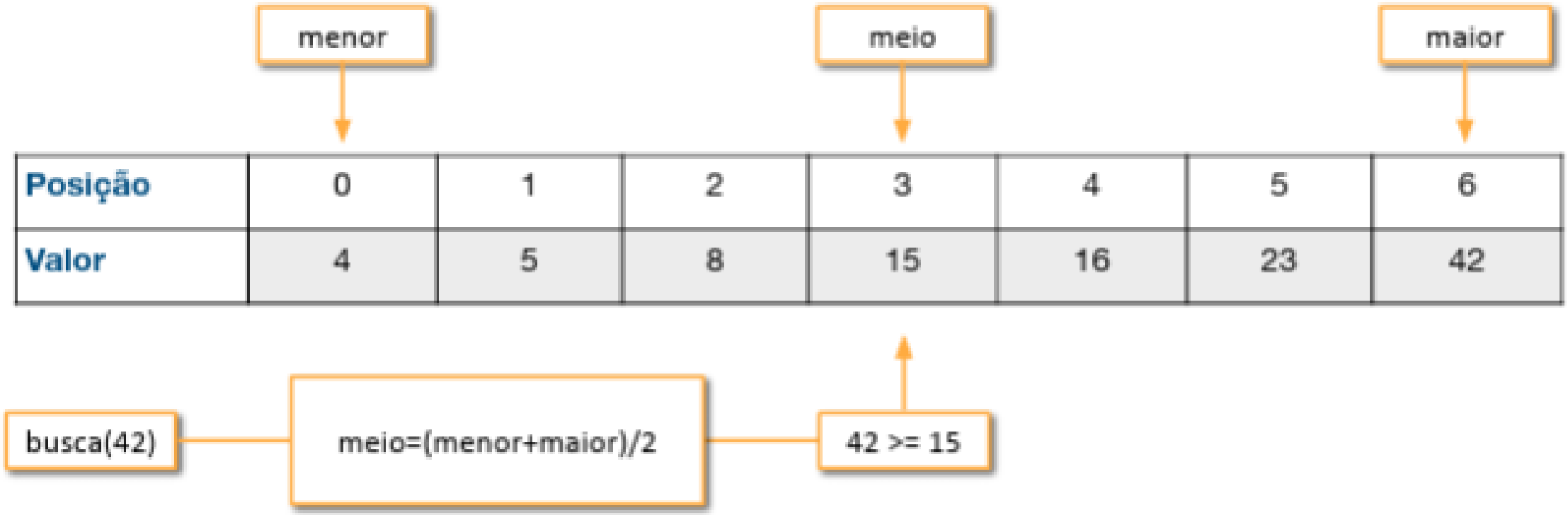
Objetivo:

- Buscar por valores em vetor ordenado (classificado)
- Continuamente dividir o espaço de busca pela metade

Suposição: Vetor ordenado

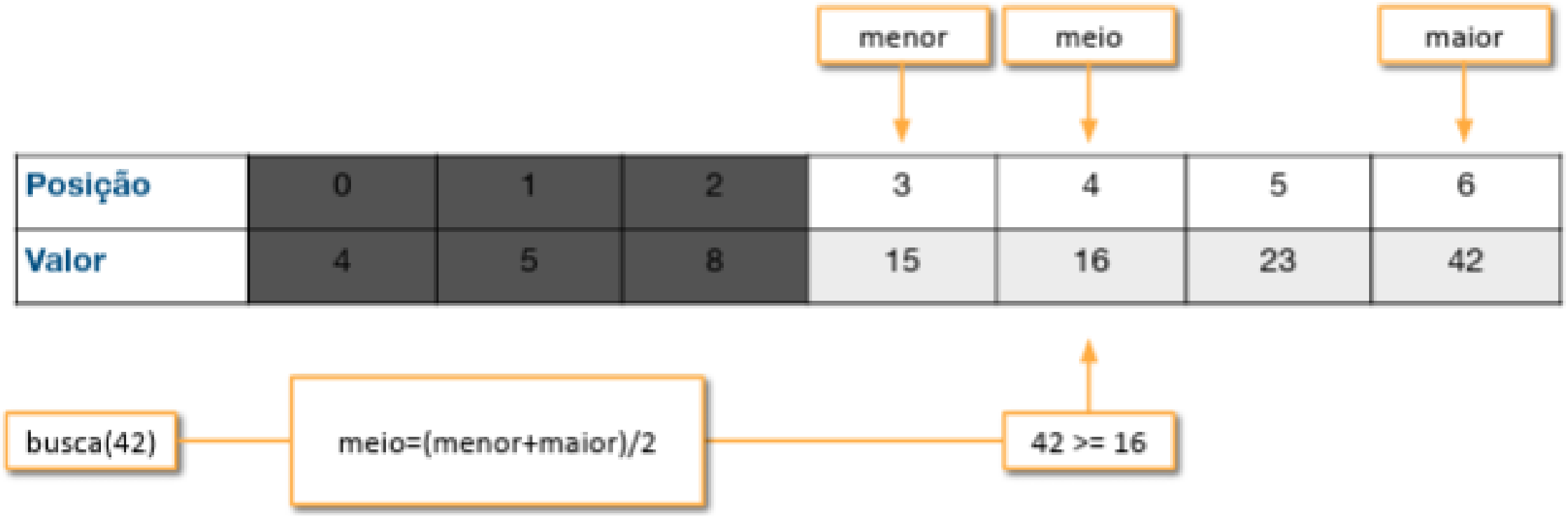
Busca Binária

Considere o conjunto de dados abaixo (ordenado):



Busca Binária

Considere o conjunto de dados abaixo (ordenado):



Busca Binária

Considere o conjunto de dados abaixo (ordenado):

menor/meio

maior

Posição	0	1	2	3	4	5	6
Valor	4	5	8	15	16	23	42

busca(42)

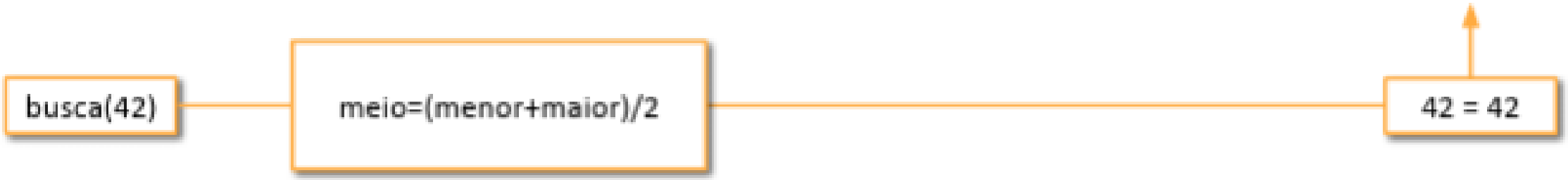
meio=(menor+maior)/2

42 >= 23

Busca Binária

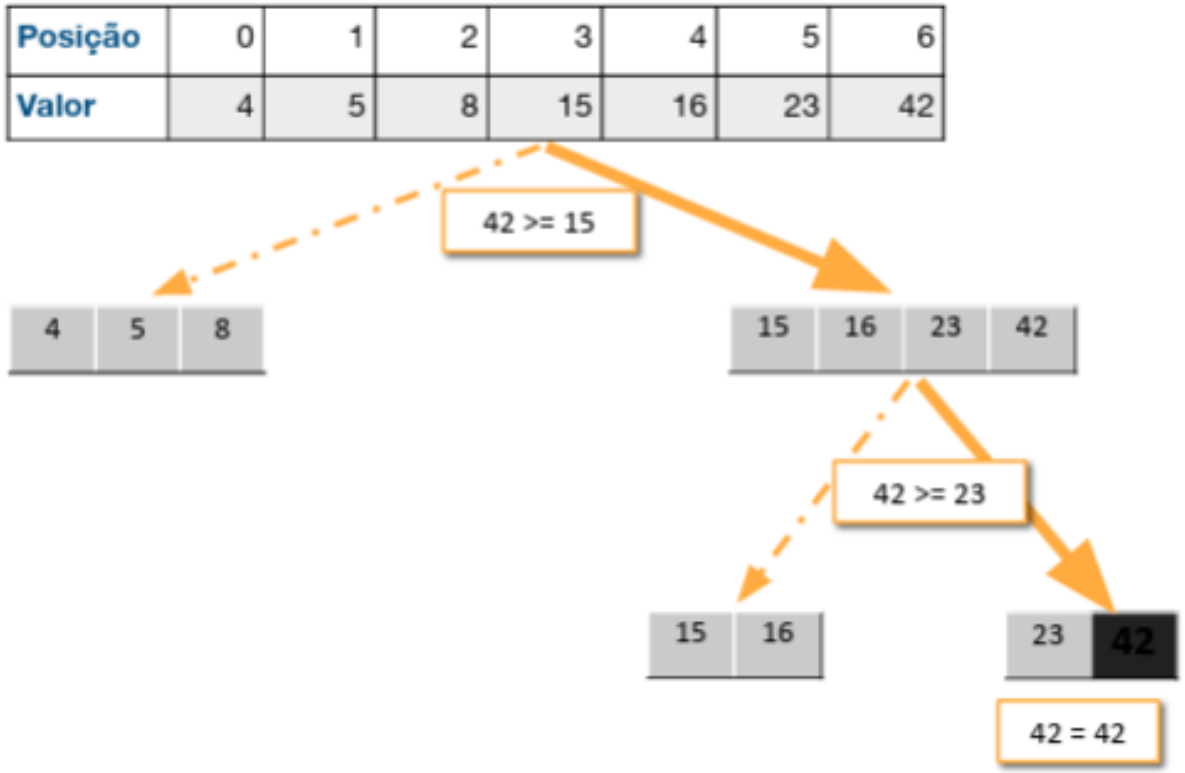
Considere o conjunto de dados abaixo (ordenado):

Posição	0	1	2	3	4	5	6
Valor	4	5	8	15	16	23	42



Busca Binária

Resumindo:



Exercício

- Implemente o algoritmo de busca binária para listas.

Mais exercícios: Sphere online judge

- ID 811 – Quermesse
- ID 813 – Saldo de Gols

Até a Próxima Aula!